

Session 5 Homework — Two More Endpoints, One Small Reconnection

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

This extends the same B4F Hub project. It is **not** a separate project — it continues from where Session 5 ended: the backend now has `GET /api/opportunities/:id` and `GET /api/posts/:id`, `PORT` is read from `process.env`, and every request is logged by a small middleware. This homework adds two more real endpoints to the same server, and reconnects one small piece of the frontend to one of them — the same shape as Session 5's own CORE and teamwork, applied one more time each.

Goal

Add a no-parameter endpoint, a parameterized endpoint with a genuinely appropriate status-code decision, and one small piece of frontend UI that actually uses one of them.

What to build

Two new endpoints in `server/index.js` — `GET /api/health` (CORE) and `GET /api/opportunities/type/:type` (STRETCH) — and, in `client/src/api.ts` and `client/src/components/Navbar.tsx`, one small piece of UI consuming the first of them (CHALLENGE).

CORE (required) — GET /api/health

1. Add `GET /api/health` to `server/index.js`. It takes no parameters and returns `{ status: "ok", opportunities: <count>, posts: <count> }`, where the counts come from the server's own in-memory `opportunities / posts` arrays.
2. This endpoint always succeeds with status `200` — there is nothing about it that can be a genuine `400 / 404`. If you find yourself writing a failure case for it, you've added something this endpoint doesn't need.

Test it directly with curl (or Postman/Thunder Client) before touching the frontend: `curl http://localhost:3001/api/health` should return the counts immediately.

STRETCH (required) — GET /api/opportunities/type/:type

1. Add a constant listing this app's real opportunity types (`job`, `internship`, `scholarship`, `volunteer`) — the exact same four values `client/src/types.ts`'s `Opportunity["type"]` and this app's own `TYPE_LABELS` already use.
2. Add `GET /api/opportunities/type/:type` to `server/index.js`. It reads `:type` from `req.params`, and:

- if `:type` is **not** one of the four real types, respond `400` with an error message naming the valid values;
 - otherwise, respond `200` with a JSON array of every opportunity whose `type` matches — an **empty array is a valid, successful result**, not an error.
3. Register this route **before** `GET /api/opportunities/:id` in the file — a route registered after it would never be reached, because Express would try to match `type` itself as an `:id` value first.

The status-code decision is the actual point of this tier: "no opportunities of this type right now" and "this isn't a real type at all" are two different problems, and they get two different status codes.

CHALLENGE (required) — the Navbar shows it's alive

1. In `client/src/api.ts`, add a small `ApiHealth` type and a `fetchApiHealth()` function calling `GET /api/health`, following the exact same `fetch + response.ok + JSON-cast` shape every other function in that file already uses.
2. In `client/src/components/Navbar.tsx`, call `fetchApiHealth()` once, when the Navbar first mounts (a plain `useState / useEffect` pair — this data belongs to exactly one component, so it does not need Redux), and render a small line under the tagline once it resolves, reading something like `API: 20 opportunities · 18 posts`.
3. If the fetch fails, the Navbar must still render everything else correctly — this line is a small extra, not a required piece of the page, so a failure here should be caught and quietly ignored, not shown as an error banner.

Rules

Allowed — everything taught by the end of Session 5: `app.get`, `req.params`, `res.status(...).json(...)`, `process.env`, the request-logging middleware pattern, plain `useState / useEffect` data fetching (the same shape `OpportunityDetailPage` already uses), and the existing `ALLOWED_CATEGORIES`-style validation pattern already in `server/index.js`.

Not yet (do not use these) — a database of any kind, `req.query`, any new npm package, Redux for this data (a new slice, a thunk, RTK Query), authentication, `.env` files, CORS configuration, or any middleware beyond the one already built in class. If you find yourself reaching for one of these, you have left the scope of this homework.

Do not modify the existing `GET /api/opportunities/:id`, `GET /api/posts/:id`, or the logging middleware — this homework only adds to the server, it never changes what already works.

Test it

- `curl http://localhost:3001/api/health` returns the real counts, and the numbers match what `/opportunities` and `/community` actually show in the app.
- `curl http://localhost:3001/api/opportunities/type/volunteer` returns `200` and an array (even if it's currently empty for some types — check at least one type that has real matches and one that might not).

- `curl -i http://localhost:3001/api/opportunities/type/banana` returns `400` and a clear error message naming the real types.
- With both servers running, load the app: the Navbar shows a live summary line matching `/api/health`'s own numbers, without needing a page reload to appear.
- Stop the backend, then reload the frontend: the Navbar still renders everything else correctly — no crash, no stuck loading state, just no summary line.
- `npm run build` and `npm run lint` (in `client/`) both pass with 0 errors. `node --check index.js` (in `server/`) reports no syntax errors.

Done when

- CORE, STRETCH and CHALLENGE are all implemented — all three are required.
- `GET /api/health`, `GET /api/opportunities/type/:type`, and the Navbar's live summary line all work exactly as described above.
- The type-filter endpoint correctly distinguishes an invalid type (`400`) from a valid type with no current matches (`200` , empty array).
- The Navbar's data fetch is a plain component-local `useState / useEffect` , not a new Redux slice.
- `npm run build` and `npm run lint` pass with 0 errors; `node --check index.js` passes.

Working time

Expect about **2 to 3 hours** — CORE is small and quick; STRETCH's real work is getting the two status codes right, not the route itself; CHALLENGE is mostly familiar (the same `fetch` shape used throughout this project), with the new part being deciding where this state belongs.

Submitting

Commit and push to your own `student-<your-github-username>` branch, as usual. Do not push to `main`.